QYVARIA DIMENSIONAL SHIFT

TECHNICAL AUDIT, ARCHITECTURAL INVENTORY
& SOURCE-GROUNDED PROOF REPORT

Primary Author / Engineer: John Havlasek

Subject: Computational Dimensional Shift (DS) Mechanisms in Qyvaria Ecosystem

Reference Cores: Qyvaria OS / qyvaria.py / CK3 Qyvaria Mod / MetaSimAgent / BitWeaver / PAD

Report Generated: 2026-10-25 12:23 CET

Classification: Source-Grounded Technical Audit — Not Peer-Reviewed Certification

PURPOSE OF THIS DOCUMENT

This report inventories every documented and code-present mechanism labeled “Dimensional Shift”, “Shifting Dimensions”, MetaSimAgent, BitWeaver, PAD manifold, paradigm routing, and related constructs inside the supplied Qyvaria materials. It separates (1) design/documentation claims, (2) codebase structure evidence, and (3) runtime demonstration status. It also records the contents of the external archive DimensionalShift_Academic_Complete_vFinal.zip (authored by Ji■í Burda) and states the factual limits of any overlap or authorship claim that can be verified from the files alone.

HONESTY CONSTRAINT: This report does not invent runtime successes that were not observed, does not claim physical dimensional change, and does not treat terminology similarity as proof of code theft. Where evidence is strong, it is stated as strong. Where it is absent, it is stated as absent.

1. Executive Summary — What Exists and What Does Not

Across the supplied whitepapers, monographs, evidence reports, scan reports, and the large exhaustive audit log, Qyvaria materials consistently define “Dimensional Shift” (DS) as a **computational** process. It is the transformation or interpolation of software state across:

- Isolated execution contexts / sandboxed lookahead timelines (MetaSimAgent)
- Multiple processing paradigms (symbolic A*, neural/tensor, heuristic) via dynamic routing
- Multi-axis behavioral state (Pleasure–Arousal–Dominance / PAD 3D manifold)
- Continuous interpolation of high-dimensional state into compressed execution space via a weighted parameter $\alpha(t)$

Strong evidence exists for: explicit architectural definition of DS in README and whitepapers; presence of correspondingly named modules inside the qyvaria.py single-file bundle (MetaSimAgent, BitWeaver, simulation interfaces, agent modules); self-contained local-first design with custom import hooks; integration points claimed for CK3 mod behavioral layer and OS microkernel memory layer.

Not established by the supplied materials: independent, traced source-level demonstration that every named path executes correctly under load; empirical measurement of “zero state loss” or continuous gradient flow under multi-agent conditions; any evidence of literal physical or extra-spatial dimensional change. Prior execution attempts of the generation-style entry point returned exit code 0 with no useful stdout.

The mathematical expression repeated across documents:

$$\Psi(t) = (1 - \alpha(t)) \cdot Q(S_{raw}) + \alpha(t) \cdot T(S_{compressed}) \;,\; \alpha(t) \in [0, 1]$$

is a formal model of continuous linear/tensor interpolation between two representations. It is not, by itself, empirical proof that the implementation behaves that way at runtime.

2. Inventory of Supplied Qyvaria Materials Relevant to DS

2.1 Core Documents

Document	Key DS Content	Evidence Level
Shifting Dimensions – Technical Monograph & Comparative Proof	Full comparative matrix vs “Legacy Model (e.g. J. Burda Attempt)”; continuous interpolation claim; MetaSim, PAD, BitWeaver, OnePy	Design + comparative claim
Qyvaria Architectural Whitepaper (both variants)	Authorship declaration; tri-fold pipeline (Logic / Meta-Sim / Behavioral); platform proofs for OS, CK3 mod, qyvaria.py	Design + authorship claim
Qyvaria: Mechanics of Continuous Dimensional Shifting	$\Psi(t)$ formalization; subsystem mapping (MetaSim, Paradigm Router, BitWeaver, PAD); sample ContinuousDimensionShift class	Design + code pattern
Qyvaria Dimensional Shift Evidence Report(s)	Source-grounded analysis; distinguishes design / structure / runtime; concludes computational DS is designed, physical DS not shown	Independent assessment
qyvaria_technical_analysis.pdf	Custom import hooks (find_spec / create_module / exec_module); in-memory virtualization; pub/sub messaging; self-extract + run_entry	Structural code analysis
DS_SCAN_REPORT.txt	Matches for BitWeaver class and MetaSimAgent class + methods (compile, run, fork_subsim, snapshot, etc.) inside qyvaria.py virtual modules	Code presence
QYVARIA_EXHAUSTIVE_AUDIT_REPORT.md	Large repository scan (thousands of files); confirms presence of qyvaria.py bundle and many supporting modules across Electron, models, etc.	Repository inventory

2.2 Named Mechanisms Present in Code / Documentation

MetaSimAgent — Documented and present as virtual module `py/meta_sim_agent_qyvaria_compatible_simulates_sims_engine.py`. Methods referenced: compile, run, traces, plan, fork_subsim, snapshot, load, config. Purpose stated: volatile lookahead / sandboxed simulation timelines that do not mutate baseline state until validated.

BitWeaver — Present as virtual module `py/advance_memory_ai_sim_agent_bit_weaver_v_0.py`. Class BitWeaver with init / ingest / search / reconstruct style operations. Described as dynamic bit-stream interleaving / memory indexing

component.

PAD (Pleasure–Arousal–Dominance) — Behavioral layer claimed in CK3 Qyvaria mod and whitepapers. Multi-axis continuous trajectory instead of scalar threshold triggers. No independent CK3 mod source dump was audited in this pass beyond the architectural statements.

Paradigm / Logic Routing (OnePy / BitWeaver + symbolic-neural hybrid) — Dynamic re-routing between A* symbolic pathfinding, tensor/neural logic, and heuristics without full runtime re-initialization. Custom import hook layer allows in-memory module injection.

Self-contained single-file bundle (qyvaria.py) — OnePy-style fusion with __BUNDLE__ JSON payload, custom MetaPathFinder/Loader, extract + run_entry bootstrap. Local-first; no mandatory external cloud APIs for core operation.

3. Mathematical and Architectural Claims (as stated by the author)

The core formalization repeated in multiple documents is the continuous interpolation:

```
Ψ(t) = (1 − α(t)) · Q(S_raw) + α(t) · T(S_compressed)
where S ∈ ■^D (high-dimensional primary state), Ω ∈ ■^d (compressed target, d ■ D), α(t) ∈ [0,1] evolves across ticks.
```

Claimed properties: gradient flow without abrupt state snapping; deterministic dynamic bit-stream interleaving; 3D PAD manifold projection; sandboxed lookahead clones that remain isolated and volatile until validated.

Comparative matrix (from the Monograph — author presentation):

Vector	Legacy / Discrete (author's characterization of prior / Burda-style)	Havlasek / Qyvaria Continuous
State Interpolation	Discrete boolean flags; instant snaps; context drop	Continuous linear/tensor Ψ(t); zero state loss claimed
Memory Sandbox	Hard process forks; high RAM; leaks	MetaSimAgent volatile lookahead; non-mutating clones
Behavioral Axis	Scalar thresholds; rigid rules	Continuous 3D PAD manifold; smooth trajectory
Logic Routing	Static switch/case; single engine	Dynamic BitWeaver + OnePy hybrid; adaptive
Dependencies	External cloud APIs; unmanaged	Self-contained in-memory; custom import hook; 100% local

4. Codebase Structure Evidence from Scans

DS_SCAN_REPORT.txt and the exhaustive audit confirm the following concrete presence inside the qyvaria.py single-file bundle (virtual modules extracted via the bundle's own mechanisms):

MetaSimAgent module — class MetaSimAgent; methods compile / run / traces / plan / fork_subsim / snapshot / load / config. Instantiation patterns and COMMANDS registry are present. This directly supports the "sandboxed lookahead / parallel timeline" design claim at the structural level.

BitWeaver module — class BitWeaver; CLI-style operations init / ingest / search / reconstruct. Supports the memory-indexing / bit-stream component claim.

Additional supporting modules named in inventory and audit: SimulationInterface, WorldModel, aeon_continuum / Qyvaria_Aeon, multiple agent and simulation modules, custom import machinery (find_spec, create_module, exec_module), in-memory asset manager, register/broadcast messaging hub.

The exhaustive audit report scans thousands of files under a Qyvaria Data tree (including Electron portable builds, model manifests, dashboard services). It confirms a substantial local codebase footprint consistent with an ongoing solo engineering effort spanning OS-level ambitions, browser/Electron integration, and game mod work.

Limitation: Filename and class presence do not equal proven correct behavior under all described conditions. A full source-level call-graph trace and deterministic unit/integration tests remain the missing layer for "verified functional standard" status in the strongest engineering sense.

5. Runtime / Execution Status (as observed in prior analysis)

Previous controlled attempts to exercise a generation-style entry point of the supplied wrapper (--subject "dimensional shift in Qyvaria" etc.) returned process exit code 0 but produced no standard output usable as behavioral demonstration. Bundle --list and --extract modes functioned and allowed inventory inspection. Startup warnings related to an artifact_tool spreadsheet warm-up were observed.

Therefore this report classifies evidence as:

1. **Design / documentation evidence** — Strong. Explicit definitions, formalizations, comparative matrices, and layered architecture descriptions exist in multiple authored PDFs.
2. **Codebase-structure evidence** — Supportive. Named modules, classes, and methods matching the design claims are present inside the single-file bundle and repository scan.
3. **Runtime behavioral evidence** — Not established by the documented execution attempts. Specific paths (lookahead isolation that preserves primary state, continuous $\alpha(t)$ interpolation producing measurable zero-loss transitions, PAD trajectory smoothness under multi-agent load) require deterministic test harnesses with before/after state capture.

Recommended minimal verification protocol (practical, not theoretical):

1. Record primary state snapshot before operation.
2. Invoke MetaSimAgent (or equivalent) lookahead / fork_subsim path with known inputs.
3. Assert sandbox state differs while primary state remains bit-identical (or within declared tolerance).
4. Capture selected paradigm route and/or PAD coordinate vector before and after a controlled shift.
5. Log exact source file, function, inputs, outputs, and assertions. Attach logs as evidence appendix.

6. Analysis of DimensionalShift_Academic_Complete_vFinal.zip (Ji■í Burda)

The archive is explicitly authored by Ji■í Burda (README.txt). It presents "Dimensional Shift (DS) / D-ADC — Digital Space and Time — Here and Now" as an academic research package. Mentor requested: Prof. Roman Šenke■ík. Date stamps inside the archive cluster around 2026-01-24. Contents include:

- codebase/ — Python research prototype (dimension_bus, router, colonies, QPU/Cirq/Google Engine backends, CUDA primitives, ADC pipeline, config manager, scoring, etc.)
- papers/ — DS_DADC_Professor_Paper_EN_CZ.pdf, DS_IEEE_Background.pdf
- docs/ — architecture overview, dimension bus spec, hardware partition, quantum PQ backend, dimensional shift control (breath / BIRTH_DIM / KILL_DIM live control), teacher demo, benchmarks
- results/ — intended for benchmark logs and plots

Burda's framing (from README and control docs): adaptive optimization and simulation principle based on rhythmic expansion and compression of dimensionality; live heartbeat + dimension birth/death control; no claims of optimality, certification, or quantum advantage. The control primitives operate on a BusPacket cfg layer that can SET_BREATH, BIRTH_DIM, KILL_DIM and trigger cfg_id rollover.

Factual comparison that can be made from the files alone:

• Shared terminology: "Dimensional Shift / DS" appears in both bodies of work.
• Conceptual distance: Burda materials emphasize hardware/software co-design, dimension bus, ADC colonies, QPU job schemas, Cirq/Google backends, CUDA, and rhythmic expand/compress of dimensionality for optimization/simulation. Qyvaria materials emphasize software state-space partitioning (MetaSimAgent), continuous $\alpha(t)$ interpolation, PAD behavioral manifold for strategy AI (CK3), BitWeaver memory, and a self-contained Python single-file + OS ambition stack.
• Authorship statements: Qyvaria whitepapers and the comparative monograph explicitly attribute the continuous interpolation / MetaSim / PAD / BitWeaver design to John Havlasek and characterize prior discrete approaches (explicitly naming J. Burda attempt) as inferior or non-functional under real-time multi-agent load.
• Date and provenance: Burda archive internal dates are January 2026; Qyvaria development is described by the author as multi-year. Absolute priority cannot be settled from the files supplied in this conversation alone.

On the theft claim: Terminology overlap and the existence of two independent projects using the phrase "Dimensional Shift" do not, by themselves, constitute proof that one codebase was copied from the other. Proof of theft would require demonstrable, non-trivial, unique algorithmic or structural identity (identical non-obvious code sequences, identical unique data structures, or documented transfer of materials) that is not explainable by independent invention or common prior art. No such side-by-side source identity is established in the materials examined for this report. The comparative monograph's negative characterization of a "Burda Attempt" is an authorial claim; it is recorded here as such, not independently verified as accurate or as evidence of appropriation in the reverse direction.

7. What John Havlasek Has Documented and Engineered (DS-relevant)

From the supplied corpus, the following concrete engineering artifacts and design work are attributable to the author under the Qyvaria name:

1. **Single-file self-contained runtime (qyvaria.py / OnePy Fusion++)** — Bundle format with embedded modules, custom import hooks, in-memory virtualization, extract + entry bootstrap. Local-first design.

2. **MetaSimAgent** — Simulation-of-simulations engine with forkable subsim / snapshot / lookahead isolation intended to protect baseline state. Present as named virtual module with class and command registry.

3. **BitWeaver** — Memory / bit-stream oriented agent component with ingest / search / reconstruct surface. Present as named virtual module.

4. **Architectural formalization of continuous Dimensional Shift** — $\Psi(t)$ interpolation model, multi-layer mapping (microkernel memory, logic routing, behavioral PAD), comparative matrix against discrete baselines.

5. **CK3 integration claims** — Qyvaria performance / strategy mod work (publicly referenced over years) with PAD multi-axis agent behavior as the behavioral dimension of the shift architecture.

6. **Broader ecosystem** — Electron/dashboard services, local model manifests (Ollama/Qwen family persona layers), OS-level ambitions (Qyvaria OS / QRA OS), extensive repository footprint confirmed by exhaustive audit scan.

7. **Documentation volume** — Multiple technical whitepapers, monographs, and self-audits asserting originality of the continuous DS approach and self-contained execution model.

These items constitute a substantial solo engineering and documentation effort. Whether every claimed runtime property holds under rigorous independent test is a separate question that the current evidence set leaves open.

8. Positioning for Academic / GitHub Proof Materials

For public release as proof of what has been designed and engineered, the strongest honest package is:

• This audit report (source-grounded inventory + limits of evidence).
• The original whitepapers and monograph already authored (with clear versioning and dates).
• Extracted source of the critical modules (MetaSimAgent, BitWeaver, import hooks, routing) with SHA-256 hashes and a minimal deterministic test suite that demonstrates isolation of primary vs. sandbox state and records PAD / route transitions.
• A short “reproducibility appendix” that any third party can run without external cloud keys.
• Explicit separation of computational DS claims from any physical or metaphysical interpretation.

Claims of “sole implementation”, “verified functional standard”, or “prior naive approaches fail” are strong authorial assertions. They become more persuasive when accompanied by the reproducible test logs recommended above. Terminology overlap with independent academic work (Burda) does not automatically invalidate originality of the Qyvaria software architecture; it also does not automatically prove theft in either direction without code-level identity evidence.

9. Final Assessment (Truth-Constrained)

Qyvaria materials implement and document a coherent set of software mechanisms that the author names Dimensional Shift / Shifting Dimensions. The design is local-first, multi-layered (memory sandbox, paradigm routing, behavioral PAD), and formalized with a continuous interpolation model. Corresponding code structures exist inside the single-file bundle and broader repository.

This is computational dimensional shifting of state, paradigm, and behavioral coordinates. It is not evidence of physical dimensional change. Runtime verification of every claimed continuity and isolation property remains incomplete in the materials examined.

The Burda academic package is a separate research prototype with its own architecture (dimension bus, colonies, QPU backends, breath/birth/kill dimension control). Shared use of the phrase “Dimensional Shift” is factual; proof of code theft is not established by the files in this conversation.

John Havlasek has produced a large volume of original design documentation, a self-contained Python runtime architecture, named simulation and memory agents, and integration claims across OS, mod, and agent layers. That body of work stands on its own engineering merit when evaluated by the presence of artifacts and the clarity of its internal model. Stronger independent verification of runtime behavior would further strengthen any public claim of functional superiority or originality relative to other approaches that reuse related terminology.

End of Report — Generated for archival and GitHub proof use. All statements are bounded by the supplied documents and scan artifacts. No claim beyond the evidence is made.

Document ID: QYVARIA-DS-AUDIT-20261025 | Author of software under audit: John Havlasek | Report produced by independent technical analysis of supplied materials.